Quant: charts, strategies, backtests, alerts

Trunk section: §6.8 Quant.

Talks to: Sources (§6.1: bar feeds); Registers (§6.6: R1 and the shortlist as backtest universes, R3 outcomes); Research data (§6.5: CANSLIM facts as of a day, for the backtest gate); the Market (§6.4: market states as of a day); the Journal (§6.9: the chart of a trade, and backtests exported as trades); Platform (§6.10: login, scheduling, backups).

Built on: the whole of legacy/qp/: LIBRARY and CATALOG (71 verified primitives) with their code in library/, STRATEGIES, BACKTEST, ALERTS-AND-PRINTS, PAGE, DATA-FEEDS, SCALPS, SCALPS_SPEC, PINE, KNOWN-ISSUES.

In one line: quant is the best-built part of the old desk. Its library, engine and backtest carry over almost unchanged; what changes is where it reads its data, the one fill model it shares with alerts, and that it no longer hosts the O'Neil chain.

Contents

1. What quant does
2. The legacy design
3. What failed
4. The new design
5. Contracts
6. How the rebuild is proven

---

1. What quant does

Job	Question
Chart	what does this stock look like, with indicators I can trust?
Build	can I describe this setup as rules, by clicking?
Backtest	if I had traded these rules on the stocks my tools found, on the days they found them, what would have happened, honestly?
Alert	is one of my saved strategies firing now on today's lists?
Print	let me review a day's (or a list's) charts, with the numbers behind them

---

2. The legacy design

2.1 The library: one rule

No market math exists anywhere except the primitive library. Every indicator is a registered function in one registry; the verification tool, the chart, the strategy engine, the backtest and the alerts all call the same function. A primitive is **trusted only once a person has compared it with TradingView and approved it**: the approval (who, when, the git commit, notes) is recorded in `approvals/approvals.json`; the catalogue is generated, never hand-written.

71 primitives, all approved:

Group	Count	Primitives
`ma`	7	sma, ema, wma, rma, vwma, hma, pine_5day
`vwap`	16	session, nday_block, nday_rolling, weekly, monthly, anchored, today_hh/ll, week_hh/ll, swing_hh/ll, last_hour_hh/ll, gap, stdev_bands
`volatility`	6	true_range, atr, atr_daily, stdev, bb, bb_ema
`osc`	1	rsi
`extremes`	2	highest, lowest
`levels`	29	day, previous day, today, pre-market, post-market, weekly, monthly, Monday, yearly opens, highs and lows
`structure`	2	pivot_high, pivot_low
`pivots`	1	floor (P, R1–R3, S1–S3)
`candle`	4	body, upper_wick, lower_wick, bar_range
`trend`	1	slope
`volume`	2	avg_volume, rel_volume

NaN means “not known yet”, never zero. **Seven deliberate differences from Pine**, each stated: VWAPs default to regular hours; two N-session VWAPs (block and rolling); `last_hour_*` stops at 16:00; floor pivots on regular-hours sessions; nothing shown before a VWAP’s first anchor; weekly, monthly and yearly levels regular-hours only (`weekly_open` is Monday 09:30); daily bars treated as the regular-hours aggregate.

2.2 The strategy grammar

A strategy is a document: name, side, stage (`development` or `ready`, set by hand), tools (its default universe), entry and exit rule groups, a risk block, an optional exit plan.

Operand	Gives
`primitive`	a verified indicator (with `sub` for one line of a multi-line one)
`price`	close, open, high, low, hl2, hlc3, ohlc4, volume
`time`	the bar’s New York minutes, hhmm, hour, minute
`const`	a number
`expr`	add, sub, mul, div of two operands, nested (÷0 gives no value)
`trade`	exit rules only: the position’s entry, bars held, unrealised %

Modifiers on any operand: `offset` (n bars ago), `hold` (carry the last value forward, before the offset), `symbol` (read from another symbol, aligned without look-ahead). Coarser timeframes show only the last **completed** bar.

14 operators: gt, lt, ge, le, eq, neq; gt_pct, lt_pct (by at least a %); cross_above, cross_below; rising, falling (over `lookback` 12, at least 75 % of non-zero changes that way, optional `min_pct`); bounce_up, bounce_down (wick within 0.15 % of the level, close back on its side in the top 60 % of the range). **Any missing value makes a rule false.** Timing: `for_bars` (all of the last N), `within_bars` (any of the last N), `offset` (true N bars ago). **Groups:** AND, OR, ATLEAST k, THEN (steps in order, each within `window` bars, default 10), nested to any depth.

Entries: `edge` (one per stretch of true bars, default) or `status` (any true bar while flat). **Exits in a fixed priority:** stop (wins over a single target in the same bar; with scale-out legs a target traded through is banked first unless `stop_first`), target or next leg, the exit rule (as a status), end of day. A stop hit in profit is recorded as `trail`.

The risk block: `sl` and `tp` as % of entry, × ATR, points, or a level that **trails** an indicator (`prim`, optionally `freeze`d at the signal bar); scale-out `targets` (fraction with R-multiple or a target); entry window; entries per day; cooldown; minimum hold; maximum and minimum stop %; minimum target in dollars; `stop_first`; `breakeven_after_target` (reported, not simulated: Q-N7). **Session rules** (prop-firm style): regular-hours entries (09:30–15:50), end-of-day close, entries per day.

2.3 Fill models

Fill	Entry at	Levels from	Use
`close`	the signal bar’s close	the same close	preview; optimistic by about a spread
`next_open`	the next bar’s open	the fill	comparison
`desk` (backtest default)	the next bar’s open	**the decision bar’s close**, as a bracket order is priced	the honest backtest
`live`	the decision price	the decision close	built for the removed live engine; unused

Why `desk` exists: a WULF short on 2026-08-19 was decided at 15.37, filled at 15.24, stop 15.74: 2.00 R as planned, 1.31 R if re-priced from the fill. A gap through a stop fills at the open (the gap is charged). An unknown fill model is an error, never a silent fallback.

2.4 The exit plan and the order

The exit is **declared, never inferred**: legs, each with its own stop and target, plus a runner; **fractions summing to exactly 1** (a plan summing to 0.9 never orders a tenth of every position). The order is **one pure function**: a risk budget (`risk_usd` or `risk_pct` of the account) ÷ risk per share; caps in order (position %, account ratio, per-order value, money

left); below `min_shares` nothing is sent; one bracket per leg in whole shares, odd shares to the runner; prices in whole cents toward the entry. The backtest books exactly these orders.

2.5 The backtest

For every day in the range and every symbol in that day's universe, the **same** `evaluate()` as the preview, as of that day; one run at a time; stoppable; resumed when the page reopens; interrupted when qp restarts; every run stores a **frozen copy of the strategies it ran**.

Universe	Each day
`symbols`	a fixed list
`register`	one tool's R1 or Shortlist, as frozen that day
`tools`	the registers of the strategy's tools, unioned

Honesty rules, all on by default: as of the day (only trades entered that day; warm-up is context); **the watchlist gate** (a trade before the scanner found the stock is dropped); the desk fill; costs (`cost_bps` per side, per-share fees with a minimum); positions open at the end recorded as open, never in the win rate; coverage stated (pairs evaluated, empty feeds, per tool, per filter).

Settings: timeframe (1m … 1d, default 5m), feed, view (all hours or regular), fill, warm-up days, session rules; filters: `min_rvol` (measured at the strategy's window start), the **CANSLIM gate** (letters as known that day; API only: Q-N4), `check_shortable` (today's flag applied to past days, said so), `rank_per_day` with a metric (`vwap_extension`, `tight_stop`, `reg_score`, …; `rank_sweep.py` measures which suits), one entry per stock per day, a trades-per- day cap. **Accounting, one of:** unit (%), account (equity, risk %, position and leverage caps, size ratio, order cap, minimum shares), TTP prop firm (fees per share, fixed size, the minimum-profit rule), challenge (target before the drawdown limit, with a worst case where every open position stops at once).

The report (re-computable from stored trades): headline (trades, wins, losses, breakeven, win rate, gross, net, profit factor never ∞, expectancy, payoff, extremes), R-multiples from net, holding time, splits by side, exit reason, symbol and day, the equity curve marked at exit, drawdown in $ and % and its length, streaks, Sharpe / Sortino / Calmar over every session (flagged on short samples), exposure, sample-adequacy warnings, the borrow check, prop-firm detail. Exports: HTML report, stats JSON, trade CSV with every register column, journal CSV.

2.6 Alerts and print sheets

The signal watcher: every saved strategy on the union of today's R1 and Shortlist, every 60 seconds (09:25–16:05), each strategy inside its own window (5 minutes' grace); an alert when the entry is true on the last completed bar; one alert per strategy, symbol and bar, repeats held for 10 minutes (in memory); one bad pair never stops a cycle. It evaluates with the `close` **fill** (Q-N9). It alerts only.

Print sheets: the day-trade sheet (every register day, one chart per ticker over 04:00–20:00 with context days), the swing sheet (a daily year before the find and every session since), their CSVs (one column per indicator, warm-up empty), the cards CSV (every frozen column), and the same for any pasted list of symbol-date pairs, optionally with **trades drawn** (the journal's chart link).

2.7 The page

One page: header, tools rail, chart settings, the chart (lightweight-charts, New York time), indicators, the ⚡ strategy drawer (▶ Evaluate with markers and the levels as simulated; 🔍 test on any rule; the signal funnel per symbol and per register day), the register browser, the backtest section, alerts, print. The working rule: **never trust a piece you have not seen**: plot, test, then compose. Bundled seeds: Test, OR + VWAP 09:35 (long and short), T2 10:00 VWAP Extension (long and short), five SMB scalps (development), PM Breakout, the trader's own PML breakout; the three ready strategies also exist as Pine scripts.

2.8 Where it lives today

`~/.qp/platform.db` (strategies, backtests, seeds), JSON copies of every strategy, the Parquet bar cache, and the O'Neil chain's caches and files. Registers are read **over HTTP from each tool's port**.

---

3. What failed

ID	Fault	Prevented by
Q-N9	alerts use the `close` fill, backtests `desk`: an alert can fire on a different bar	one fill model for preview, backtest and alerts
Q-N7	`breakeven_after_target` reported, never simulated	simulated, or refused on save
Q-N4	the CANSLIM gate has no control on the page	a control, reading facts as of the day
Q-N6	some routes default to Polygon	no per-route defaults (SOURCES.md §4)
Q-N5	the nightly O'Neil chain lives here	moved to the research worker (ONEIL.md)
—	while qp is down or restarting, every ticker first seen has a blank CANSLIM reading, and a photo then freezes the blanks	quant computes no card facts
—	registers read over HTTP from every tool's port; a stopped tool needs the archive process	read from the database
—	a qp restart took 3 min 20 s (fixed)	one supervisor; a start-up check that refuses only a listening server
—	alert repeats held in memory (a restart can alert again)	alert history stored
—	each (strategy, symbol) pair fetches its own bars every cycle	a shared bar cache per cycle
Q-N1–Q-N3, Q-N8, Q-N10	out-of-date text	generated help

4. The new design

4.1 Kept as it is

- **The library**: its code (library/), the registry, the approval gate, the generated catalogue, the seven stated deviations. A new primitive is not used until approved.
- **The engine**: the grammar, the exit priority, the risk block, the exit plan, the order function.

- **The backtest**: universes, honesty rules, settings, accounting blocks, the report, frozen strategy copies.
- **The page** and the print sheets.

4.2 Changed

Change	Detail
One fill model for alerts	alerts evaluate with the strategy's own fill (default `desk`), so an alert and a backtest of the same strategy take the same bar (Q-N9)
Data from the database	registers (R1, Shortlist, R3), market states and CANSLIM facts are read **as of a day** from the database, not over HTTP; archived tools are just tools
No O'Neil computation	the research worker owns it; quant reads facts
Alerts as a scheduled job	the platform scheduler runs the watcher's cycles; alert history (strategy, symbol, bar, sent) is stored, so repeats survive a restart and alerts can be reviewed; a push to the phone as well as the page
Shared bars per cycle	one fetch per symbol per cycle, shared by every strategy
The CANSLIM gate on the page	a control, with the letters that cannot be rebuilt for a day refused before the run
`breakeven_after_target`	simulated (move the stop to entry after the first target fills), or refused on save until it is
A trade's chart	the journal sends real instants; print sheets mark them on the right bars

4.3 Where strategies live

In the database (`strategies`, `strategy_versions`): saving a strategy creates a version; a backtest records the versions it ran (replacing the frozen copies with references, while keeping a copy for old runs). Seeds become versioned migrations, as for screeners. JSON exports of every strategy are part of the nightly backup.

---

5. Contracts

Owns

Table	Key
`strategies`, `strategy_versions`	id, version
`backtests`, `backtest_trades`	id
`alerts`	strategy, symbol, bar
`primitive_approvals` (mirrors the library's `approvals.json`)	primitive, commit

Reads

From	What
sources	bars (the chosen feed)
registers	R1, Shortlist, R3, as of a day
market, research data	market states, CANSLIM facts, as of a day

Routes (quant, behind the front door at `/quant`)

The legacy routes carry over (legacy/qp/API.md: 53 routes and the live WebSocket), with the register routes reading the database and the alert routes reading stored history.

---

6. How the rebuild is proven

1. **Backtest parity.** A set of past backtests re-run on the new data path gives the same trades and numbers (the strategy versions and universes are the same).
2. **Alert parity with the backtest.** For a day, every alert's bar equals the bar the backtest of the same strategy enters on.
3. **Library.** qp's own test suite (`run_all`) and the 71 approvals carry over unchanged in CI.
4. **CANSLIM gate.** A gated backtest reads the same letters the cards showed that day.